

C++ STL CHEAT SHEET (DSA + Interviews)

1. CONTAINERS

1.1 Vector

```
vector<int> v;  
• push_back(x)  
• pop_back()  
• insert(v.begin()+i, x)  
• erase(v.begin()+i)  
• v[i]  
• front(), back()  
• size(), empty(), clear()
```

Note: Dynamic Array

1.2 Deque

```
deque<int> dq;  
• push_back(x)  
• push_front(x)  
• pop_back()  
• pop_front()  
• front(), back()  
• dq[i]  
• size(), empty()
```

Note: Insert/Delete from both ends

1.3 Set

```
set<int> s;  
• insert(x)  
• erase(x)  
• find(x)  
• lower_bound(x)  
• upper_bound(x)  
• size(), empty(), clear()
```

Note: Sorted + Unique

1.4 Multiset

```
multiset<int> ms;  
• insert(x)  
• erase(x) // all x  
• erase(ms.find(x)) // one x  
• find(x)  
• count(x)  
• lower_bound(x)  
• upper_bound(x)
```

Note: Sorted + Duplicates

1.5 Unordered Set

```
unordered_set<int> us;  
• insert(x)  
• erase(x)  
• find(x)  
• size(), empty()
```

Note: Unique, Not Sorted

1.6 Map

```
map<int,int> mp;  
• mp[key] = value  
• emplace(key,value)  
• find(key)  
• erase(key)  
• size(), empty(), clear()
```

Note: Sorted by Key

1.7 Multimap

```
multimap<int,int> mm;  
• insert({1,10})  
• insert({1,20})  
• find(1)  
• count(1)  
• erase(1)
```

Note: Duplicate Keys Allowed

1.8 Unordered Map

```
unordered_map<int,int> ump;  
• ump[key] = value  
• emplace(key,value)  
• find(key)  
• erase(key)  
• size(), empty()
```

Note: Not Sorted

1.9 Frequency Count

```
unordered_map<char,int> freq;  
for(char c : s)  
    freq[c]++;  
(Useful for counting)
```

7. UTILITY FUNCTIONS

7.1 Min / Max

```
min(a, b);  
max(a, b);
```

7.2 Absolute Value

```
abs(x);
```

7.3 String Stream

```
stringstream ss;  
ss << "123";  
ss >> x;
```

7.4 Swap

```
swap(a, b);
```

2. ADAPTERS (CONTAINER ADAPTERS)

2.1 Stack

```
stack<int> st;  
• push(x)  
• pop()  
• top()  
• size(), empty()
```

2.2 Queue

```
queue<int> q;  
• push(x)  
• pop()  
• front(), back()  
• size(), empty()
```

2.3 Priority Queue

Max Heap (Default)
priority_queue<int> pq;
• push(x)
• pop()
• top()
• size(), empty()

Min Heap
priority_queue<int, vector<int>, greater<int>> pq;

Pair Priority Queue
priority_queue<pair<int,int>> pq;

3. PAIRS & TUPLES

3.1 Pair

```
pair<int,int> p;  
• p.first  
• p.second  
• swap(p.first, p.second)
```

3.2 Tuple

```
tuple<int,char,double> t;  
• get<0>(t)  
• get<1>(t)  
• get<2>(t)
```

4. VECTOR OF PAIRS

```
vector<pair<int,int>> vp;  
• push_back({a,b})  
• emplace_back(a,b)  
• vp[i].first  
• vp[i].second
```

Sort by First

```
sort(vp.begin(), vp.end());
```

Sort by Second

```
bool cmp(pair<int,int>& a, pair<int,int>& b){  
    return a.second < b.second;  
}  
sort(vp.begin(), vp.end(), cmp);
```

Descending

```
sort(vp.rbegin(), vp.rend());
```

5. ITERATORS & LOOPS

5.1 Iterators

```
auto it = v.begin();  
auto it = v.end();
```

5.2 Range Loop

```
for(auto x : v)  
    cout << x;
```

6.3 By Reference

```
for(auto &x : v)  
    cin >> x;
```

5.4 Iterator Loop

```
for(auto it=v.begin(); it!=v.end(); it++){  
    cout << *it;  
}
```

6. ALGORITHMS

6.1 Sorting

Ascending
sort(v.begin(), v.end());

Descending
sort(v.begin(), v.end(), greater<int>());

Lambda Comparator
sort(vp.begin(), vp.end(), [](auto &a, auto &b){
 return a.second < b.second;
});

6.2 Binary Search

```
binary_search(v.begin(), v.end(), x);  
(Array must be sorted)
```

6.3 Lower / Upper Bound

```
auto lb = lower_bound(v.begin(), v.end(), x);  
auto ub = upper_bound(v.begin(), v.end(), x);  
• lower_bound → first element >= x  
• upper_bound → first element > x
```

Index

```
int idx = lower_bound(v.begin(), v.end(), x) - v.begin();
```

6.4 Min / Max Element

```
auto mx = max_element(v.begin(), v.end());  
auto mn = min_element(v.begin(), v.end());  
  
Value:  
*mx; *mn;
```

6.5 Count

```
count(v.begin(), v.end(), x);
```

6.6 Find

```
find(v.begin(), v.end(), x);
```

6.7 Reverse

```
reverse(v.begin(), v.end());
```

6.8 Fill

```
fill(v.begin(), v.end(), x);
```

6.9 Accumulate

```
accumulate(v.begin(), v.end(), 0);
```

6.10 Next / Prev Permutation

```
next_permutation(v.begin(), v.end());  
prev_permutation(v.begin(), v.end());
```

8. STRING FUNCTIONS

Size	s.size(); s.length();
Access	s[i]; s.front(); s.back();
Substring	s.substr(start, len);
Find	s.find("abc"); s.find(ch); // Not found: string::npos
Erase	s.erase(pos, len);
Insert	s.insert(pos, str);
Replace	s.replace(pos, len, newStr);
Reverse String	reverse(s.begin(), s.end());
Sort String	sort(s.begin(), s.end());
Case Conversion	toupper(ch); tolower(ch); (Entire String: for(char &c : s) c = toupper(c);)
Conversion	stoi(str); stol(str); stoll(str); stof(str); stod(str); to_string(x);
Input with Spaces	getline(cin, s);

9. TOP 15 FUNCTIONS IN DSA

1 sort()	6 find()	11 next_permutation()
2 reverse()	7 count()	12 fill()
3 lower_bound()	8 accumulate()	13 swap()
4 upper_bound()	9 max_element()	14 stoi()
5 binary_search()	10 min_element()	15 getline()